

COS 214 PRACTICAL 5 - CAMPUSGUARD

Hannah Diedrick - u25044215

Lucian Van der Merwe - u25054262

Ajba Patel- u25316584

1. LEGACY SERVICE AND ADAPTER DESIGN (Task 1)

1.1 Legacy service to adapt

CampusGuard must control door locks across campus, but the campus already has an old access-control panel that we cannot modify. It is represented by the class LegacyDoorPanel. Its interface does not fit our system:

- doors are identified by numeric zone numbers (for example 101 or 201);
- actions are numeric opcodes (1 = lock, 2 = unlock, 3 = card-only);
- results are integer return codes (0 = success, -1 = bad opcode, -2 = unknown zone, -3 = zone offline);
- its status query also returns integers (0 = open, 1 = locked, 2 = card-only).

1.2 Adapter participants

Adapter converts the interface of an existing class into an interface that the client expects.

Target: AccessController. The interface CampusGuard wants to use. Operations: lock(areald), unlock(areald), restrictToStaff(areald), statusOf(areald). Areas are named (for example "ENG-ENTRANCE"), failures are reported with an AccessFault exception, and status is returned as an AccessLevel (open, locked, restricted).

Adaptee: LegacyDoorPanel (described above).

Adapter: LegacyPanelAdapter. Inherits from AccessController and holds a reference to a LegacyDoorPanel. For each request it (a) maps the area name to a zone number using an internal lookup table, (b) maps the action to the correct opcode, (c) calls execute() on the panel, and (d) converts a non-zero return code into an AccessFault with a readable message. statusOf() converts the panel's integer status into an AccessLevel.

Client: BuildingAccessService. It only knows the AccessController interface, never the legacy panel.

Why an adapter: the panel's interface is incompatible with what CampusGuard expects (numbers vs names, return codes vs exceptions). The adapter translates between the two, so it is a real interface translation and not just a wrapper.

1.3 Command design

Purpose: The Command pattern is used to represent operator actions as objects. This separates the object that requests an emergency action from the response component that

performs the action. This allows commands to be executed uniformly through the operator console and allows previous actions to be cancelled where appropriate.

Participants:

GoF participant	Class name	Use
Command	Command	Declares the common interface for executable commands.
Concrete Command	DispatchUnitCommand, RestrictAreaCommand, IssueAlertCommand, CancelActionCommand	Dispatches a response unit to an incident. Requests restricted access to an affected campus area. Issues an emergency campus alert. Cancels or reverses a previously issued action where possible.
Invoker	OperatorConsole	Receives a command and invokes its execution without knowing the command's implementation.
Receivers	SecurityTeam, FacilitiesStaff, CommunicationService	Perform the actual domain operations requested by the commands.
Client	Main	Creates and configures the required command objects.

Command is an abstract polymorphic base class and defines the common operation used by the invoker. OperatorConsole depends only on this interface rather than on each concrete action. Each concrete command stores the receiver and information required to perform its task. This reduces coupling between the operator interface and the emergency-response components.

1.4 Mediator:

Purpose: The Mediator pattern is used to coordinate Security, Medical, Facilities and Communication components without those components communicating directly with one another. All cross-component coordination is routed through the ResponseCoordinator.

Participants:

GoF participant:	Class name:	Purpose:
Mediator	ResponseMediator	Defines the interface through which response

		colleagues report events.
Concrete mediator	ResponseCoordinator	Contains the coordinator rules and decides which other colleagues must react to an event.
Colleague	ResponseColleague	Common base for components that communicate through the mediator.
Concrete Colleague	SecurityTeam, MedicalResponder, FacilitiesStaff, CommunicationServices	Dispatches security and reports security related events. Responds to medical coordination requests. Performs facilities and emergency-route operations. Issues emergency and internal notifications.

The response colleagues do not maintain direct references to all other response colleagues. For example, SecurityTeam does not directly call MedicalResponder or FacilitiesStaff. Instead, Security reports an event to ResponseCoordinator. The coordinator decides which other components need to react. This avoids many-to-many coupling and keeps coordination rules in one appropriate location.

Command and Mediator collaborate during a normal incident-response workflow. An operator submits a DispatchUnitCommand to the OperatorConsole. The command retrieves the relevant incident and invokes its receiver, SecurityTeam. After Security has been dispatched, SecurityTeam reports a security_dispatched event to the ResponseCoordinator. The mediator then coordinates the other response colleagues. In the demonstrated scenario, Facilities opens an emergency route, Medical prepares for the incident and the Communication service issues the relevant notification. Therefore the Command initiates the requested action while the Mediator coordinates the consequences of that action across the wider system.

1.5 State:

An Incident moves through Reported->Dispatched->Contained->Resolved and each allows different actions.

Participants:

GoF participant	Class name	Use
Context	Incident	Holds current IncidentState
State	IncidentState	Abstract: declares

		dispatch(), contain(), resolve(), name()
ConcreteState	ReportedState, DispatchedState, ContainedState, ResolvedState	Each implements its own legal transitions, illegal ones call reject

1.6 Observer:

When an Incident's state changes, other parts of the system need to know, without Incident being hard coded to call a specific class. Observer lets Incident broadcast to whatever's attached.

Participants:

GoF participant	Class name	Use
Subject	Incident	Owns observers_, calls notify() on state change
Observer	IncidentObserver	Abstract, declares onIncidentStateChanged()
ConcreteObserver	IncidentBoard	Prints the transition and records it in AuditLog

2. FACADE DESIGN (Task 1)

Facade provides a simple interface to a complicated subsystem.

Facade: LockdownFacade.

Subsystems: BuildingAccessService, NotificationService, AuditLog.

Workflow: initiateLockdown(buildingId, incidentRef)

1. Record the start of the lockdown in AuditLog.
2. Ask BuildingAccessService for the areas in the building.
3. For each area, lock it if it is an entrance or restrict it to staff if it is an interior area (through BuildingAccessService, which reaches the legacy panel through the adapter).
4. Broadcast a lockdown message through NotificationService ("all areas secured", or "PARTIAL LOCKDOWN, manual check needed" if any failed).
5. Record the result (complete or partial) in AuditLog.
6. Return a LockdownReport (areas secured and a list of failures).

This coordinates three subsystem services (BuildingAccessService, NotificationService, AuditLog). A matching liftLockdown() reopens the areas, broadcasts an all-clear (or a

partial-release warning), and records it. Both operations abort and report a failure if the building is unknown.

Complexity hidden by the facade: without it the client would need to know which areas are entrances and which are interior, call access control, notifications and the audit log in the right order, and handle each failure separately.

Independence of subsystems: the subsystem classes stay public and are not wrapped. For example, a medical team can call `BuildingAccessService::unlockArea("ENG-ENTRANCE")` directly without the facade.

Incident dispatch and coordinated response

Setup:

An incident has been registered for a fire in the Engineering Building. Security, Medical, Facilities and Communication components have been registered with the `ResponseCoordinator`.

Step 1 — Operator command

The operator requests a security response. `OperatorConsole` executes a `DispatchUnitCommand` for the incident.

Step 2 — Command execution

`DispatchUnitCommand` obtains the required incident from the incident registry and dispatches its receiver, `SecurityTeam`.

Step 3 — Mediator coordination

After dispatching, `SecurityTeam` sends a `security_dispatched` event to `ResponseCoordinator` rather than contacting the other response components directly.

Step 4 — Other colleagues react

Step 5 — Further commands

Each command operates on its appropriate receiver while related cross-component reactions continue to be coordinated through the mediator.

Patterns shown: Command + Mediator.

3. OWNERSHIP DECISIONS (Task 1)

- BuildingAccessService owns its AccessController through a `std::unique_ptr`, so the adapter is destroyed together with the service.
- LegacyPanelAdapter holds a non-owning reference to LegacyDoorPanel. The panel is owned by main and is declared before the adapter, so it outlives it.
- LockdownFacade holds non-owning references to BuildingAccessService, NotificationService and AuditLog, which are owned by main.
- AccessController has a virtual destructor, so deleting an adapter through the base interface is safe.
- OperatorConsole owns the commands stored in its command history using smart pointers. This gives each command a clearly defined lifetime and prevents memory leaks when command objects are destroyed.
- The response colleagues hold a non-owning pointer/reference to the ResponseMediator. They do not destroy the mediator.
- ResponseCoordinator holds non-owning references to the registered response colleagues. The coordinator coordinates these objects but does not own their lifetime.
- All polymorphic base classes, including Command, ResponseMediator and ResponseColleague, provide virtual destructors so that derived objects can be destroyed correctly through base-class pointers.

TASK 3: SCENARIOS

ENGINEERING BUILDING LOCKDOWN:

Setup: the legacy door panel has three zones, mapped to areas by the adapter: ENG-ENTRANCE (zone 101, entrance), ENG-LAB (zone 102, interior) and ENG-OFFICE (zone 103, interior).

Step 1 - Adapter used on its own: BuildingAccessService.lockArea, restrictArea and unlockArea are called directly. Each call goes through LegacyPanelAdapter, which translates the named area into a zone number and the action into an opcode for LegacyDoorPanel.execute().

Step 2 - Facade, complete lockdown: an operator calls LockdownFacade.initiateLockdown("ENG", "INC-1"). The facade:

1. records the start in AuditLog;
2. asks BuildingAccessService for the Engineering areas, locks the entrance and restricts the two interior areas (each request goes through LegacyPanelAdapter to LegacyDoorPanel);
3. broadcasts "LOCKDOWN in ENG: all 3 areas secured. Shelter in place." through NotificationService;
4. records the result in AuditLog and returns a LockdownReport (secured = 3, failed = 0).

Step 3 - Facade, failure path: zone 102 (the laboratory) is set offline, then liftLockdown("ENG", "INC-1") is called. The panel returns -3, the adapter throws AccessFault, BuildingAccessService logs the failure and returns false, and the facade reports the laboratory as failed. NotificationService broadcasts "PARTIAL RELEASE in ENG: manual check needed for ENG-LAB" and AuditLog records the partial result (secured = 2, failed = 1). Nothing is silently ignored.

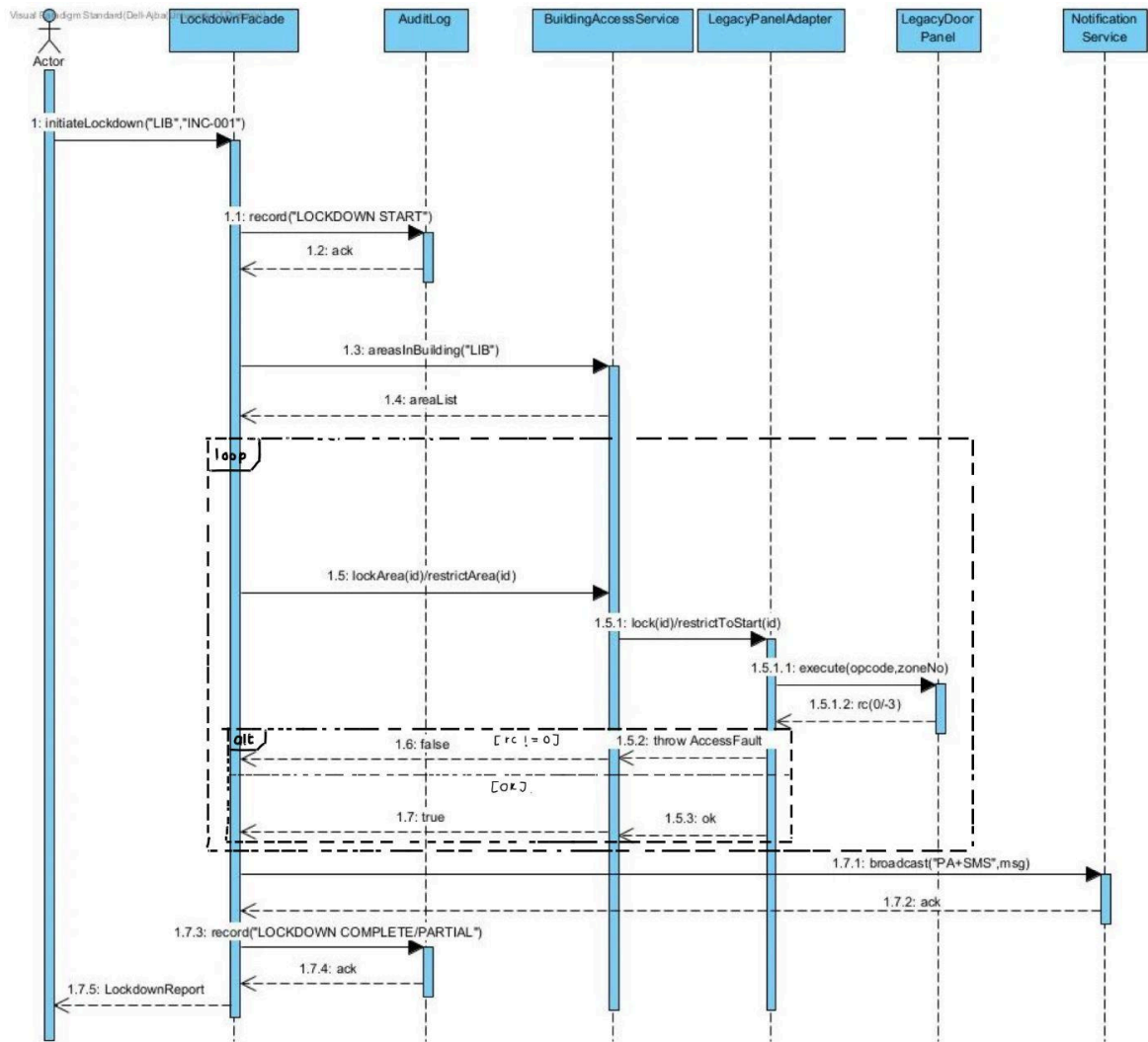
Step 4 - Invalid input: initiateLockdown("UNKNOWN", "INC-X") aborts cleanly. The facade records "LOCKDOWN ABORTED: unknown building UNKNOWN" in AuditLog and returns one failure.

Independent use: BuildingAccessService, NotificationService and AuditLog are used directly in Step 1 without the facade.

TASK 4

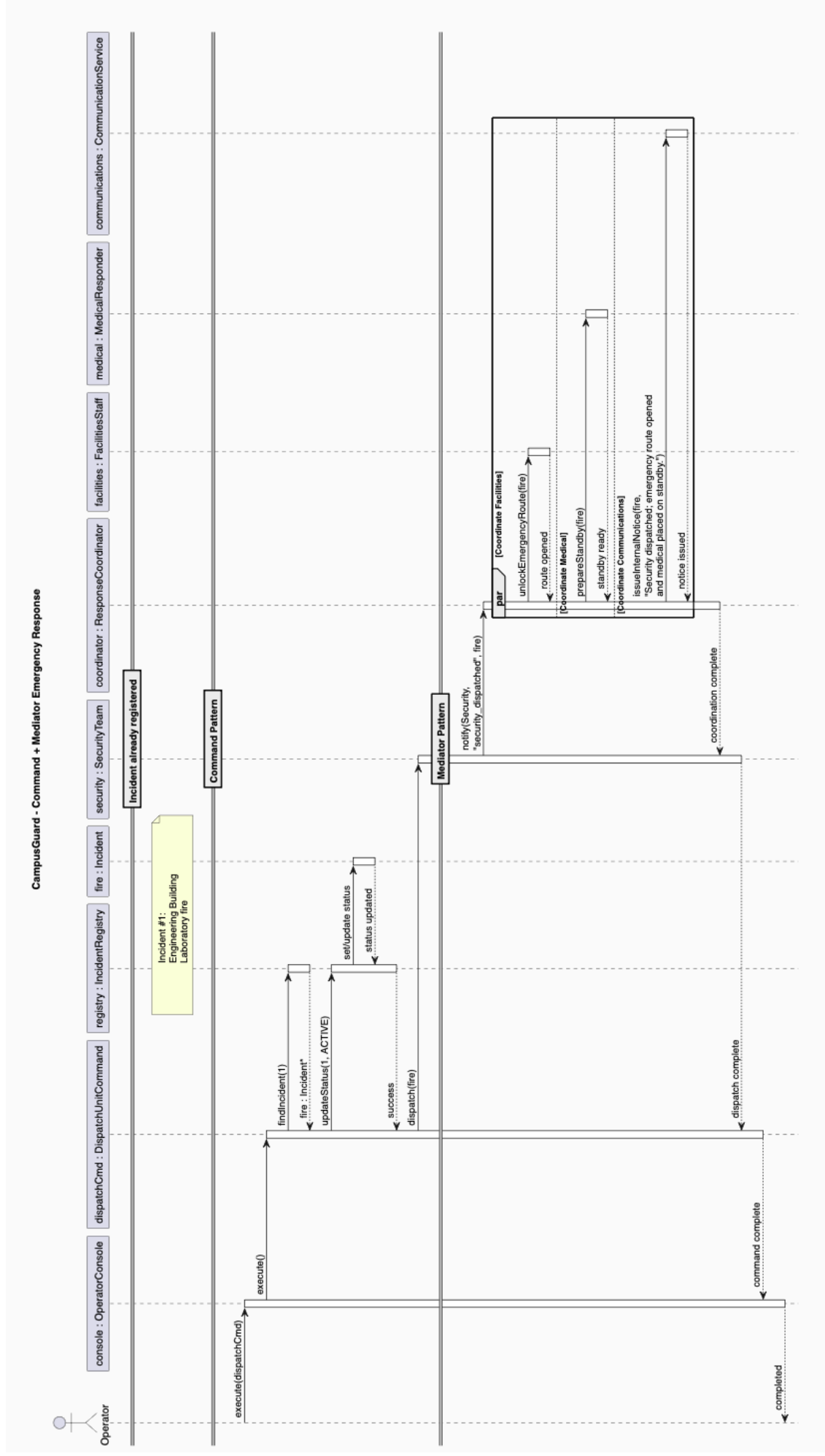
Sequence diagram: Facade with adapter

Description: the operator calls LockdownFacade.initiateLockdown. The facade records the start in AuditLog and asks BuildingAccessService for the building's areas. For each area it calls lockArea or restrictArea. BuildingAccessService calls LegacyPanelAdapter.lock or restrictToStaff, which calls LegacyDoorPanel.execute(opcode, zoneNo). If the panel returns a non-zero code, the adapter throws AccessFault and the service returns false; otherwise it returns true. After all areas are processed the facade calls NotificationService.broadcast, records the result in AuditLog, and returns the LockdownReport to the operator. The same failure handling (the alt branch) applies in liftLockdown, which is the path used by the demo's failure case.



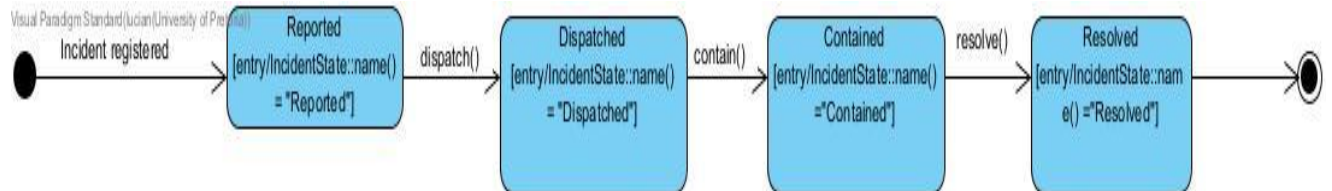
Sequence diagram: Command with mediator

The operator submits a DispatchUnitCommand through OperatorConsole. The console invokes execute() on the command. The command retrieves the incident and invokes dispatch() on SecurityTeam, its receiver. Security performs the dispatch and reports a security_dispatched event to the ResponseCoordinator. The mediator then coordinates FacilitiesStaff, MedicalResponder and CommunicationService. This diagram demonstrates the Command pattern from invoker to concrete command to receiver and then shows how the receiver's action leads into Mediator coordination.



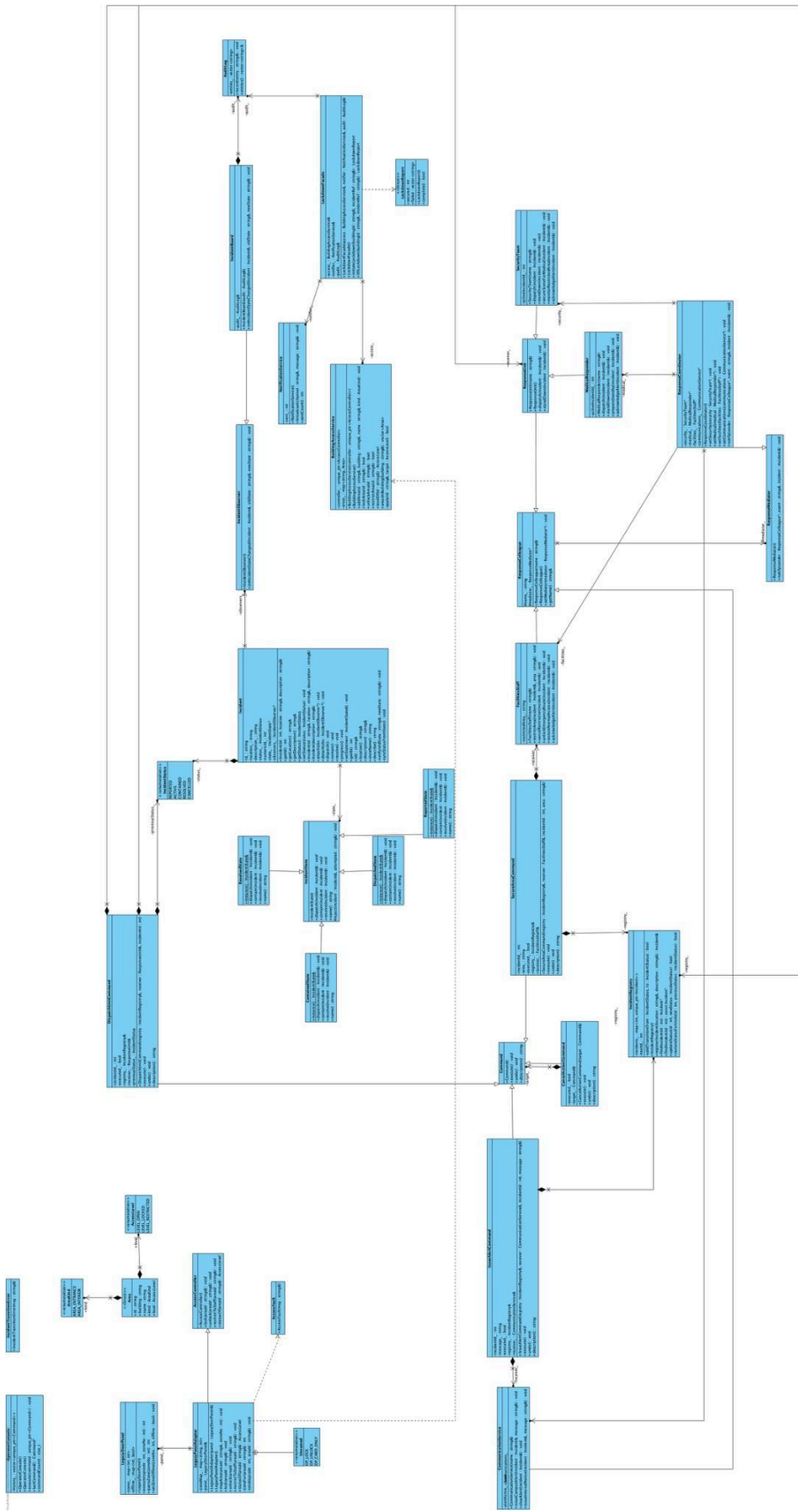
State diagram(State):

A UML state diagram showing the Incident lifecycle (Reported->Dispatched->Contained->Resolved) with each valid transition handled by a ConcreteState and every illegal transition rejected by IncidentTransitionError rather than silently changing.



UML diagram:

The class diagram represents the final implemented CampusGuard system. It contains all six GoF patterns and shows the inheritance, association, ownership and dependency relationships used by the final C++ implementation. The Command hierarchy is connected to the OperatorConsole invoker and its receivers, while the Mediator hierarchy shows ResponseCoordinator coordinating the response colleague classes.



Task 5:

```
● ajba@DESKTOP-CK66S0J:/mnt/c/Users/Dell-Ajba/Music/214 prac 1$ make valgrind
valgrind --leak-check=full --show-leak-kinds=all ./campusguard
==19544== Memcheck, a memory error detector
==19544== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==19544== Using Valgrind-3.15.0 and LibVEX; rerun with -h for copyright info
==19544== Command: ./campusguard
==19544==

=====
CAMPUSGUARD INTEGRATED TEST
=====

--- 1. STATE + OBSERVER ---
[Board] 900 (Engineering Lab): REPORTED -> DISPATCHED
[Audit #1] STATE 900 REPORTED -> DISPATCHED
[Board] 900 (Engineering Lab): DISPATCHED -> CONTAINED
[Audit #2] STATE 900 DISPATCHED -> CONTAINED
[Board] 900 (Engineering Lab): CONTAINED -> RESOLVED
[Audit #3] STATE 900 CONTAINED -> RESOLVED
Final lifecycle: Smoke detected [RESOLVED]

--- 2. MEDIATOR + RESPONSE COLLEAGUES ---

--- 3. REGISTRY + COMMAND + INVOKER ---
[IncidentRegistry] Registered incident #1 at Engineering Building: Laboratory fire

[Invoker:OperatorConsole] Executing -> Dispatch response unit to incident #1
[IncidentRegistry] Incident #1 status: REPORTED -> ACTIVE
[Receiver:SecurityTeam] Campus Security dispatched to incident #1 at Engineering Building
[Mediator:ResponseCoordinator] Event 'security_dispatched' received from Campus Security for incident #1
[FacilitiesStaff] Emergency route unlocked for incident #1
[MedicalResponder] Preparing medical standby for incident #1
[CommunicationService] Internal notice for incident #1: Security dispatched; emergency route opened and medical placed on standby.

[Invoker:OperatorConsole] Executing -> Restrict Engineering Lab for incident #1
[Receiver:FacilitiesStaff] Restricted access to Engineering Lab for incident #1
[Mediator:ResponseCoordinator] Event 'area_restricted' received from Facilities for incident #1
[SecurityTeam] Monitoring restricted area for incident #1
[CommunicationService] Internal notice for incident #1: Facilities changed access controls; security monitoring requested.

[Invoker:OperatorConsole] Executing -> Issue campus alert for incident #1
[Receiver:CommunicationService] CAMPUS ALERT for incident #1: Evacuate Engineering Building immediately.
[Mediator:ResponseCoordinator] Event 'campus_alert_issued' received from Communications for incident #1
[SecurityTeam] Emergency alert acknowledged for incident #1
[MedicalResponder] Emergency alert acknowledged for incident #1
[FacilitiesStaff] Emergency alert acknowledged for incident #1
Commands stored by invoker: 3
```

```

--- 4. COMMAND UNDO + CANCEL COMMAND ---
[CancelActionCommand] Cancelling previous action: Issue campus alert for incident #1
[CommunicationService] Cleared campus alert for incident #1
[CancelActionCommand] Re-applying cancelled action: Issue campus alert for incident #1
[Receiver:CommunicationService] CAMPUS ALERT for incident #1: Evacuate Engineering Building immediately.
[Mediator:ResponseCoordinator] Event 'campus_alert_issued' received from Communications for incident #1
[SecurityTeam] Emergency alert acknowledged for incident #1
[MedicalResponder] Emergency alert acknowledged for incident #1
[FacilitiesStaff] Emergency alert acknowledged for incident #1

--- 5. ADAPTER + BUILDING ACCESS SERVICE ---
[LegacyPanel] EXEC op=1 zone=101
[Access] Engineering Entrance (ENG-ENTRANCE) -> LOCKED
[LegacyPanel] EXEC op=3 zone=102
[Access] Engineering Laboratory (ENG-LAB) -> RESTRICTED
[LegacyPanel] EXEC op=2 zone=101
[Access] Engineering Entrance (ENG-ENTRANCE) -> OPEN

--- 6. FACADE: COMPLETE LOCKDOWN ---
[Audit #4] LOCKDOWN START building=ENG incident=INC-1
[LegacyPanel] EXEC op=1 zone=101
[Access] Engineering Entrance (ENG-ENTRANCE) -> LOCKED
[LegacyPanel] EXEC op=3 zone=102
[Access] Engineering Laboratory (ENG-LAB) -> RESTRICTED
[LegacyPanel] EXEC op=3 zone=103
[Access] Engineering Office (ENG-OFFICE) -> RESTRICTED
[Notify:PA+SMS] LOCKDOWN in ENG: all 3 areas secured. Shelter in place.
[Audit #5] LOCKDOWN COMPLETE building=ENG
Lockdown secured=3 failed=0

```

```

--- 7. FACADE FAILURE PATH ---
[Audit #6] LOCKDOWN LIFT building=ENG incident=INC-1
[LegacyPanel] EXEC op=2 zone=101
[Access] Engineering Entrance (ENG-ENTRANCE) -> OPEN
[Access] FAILED: legacy panel rejected request for 'ENG-LAB' (zone 102): zone offline
[LegacyPanel] EXEC op=2 zone=103
[Access] Engineering Office (ENG-OFFICE) -> OPEN
[Notify:PA+SMS] PARTIAL RELEASE in ENG: manual check needed for ENG-LAB
[Audit #7] LOCKDOWN LIFT PARTIAL building=ENG
Release secured=2 failed=1

--- 8. INVALID INPUT / ERROR HANDLING ---

[Invoker:OperatorConsole] Executing -> Dispatch response unit to incident #9999
[DispatchUnitCommand] ERROR: incident #9999 does not exist.
[Audit #8] LOCKDOWN START building=UNKNOWN incident=INC-X
[Audit #9] LOCKDOWN ABORTED: unknown building UNKNOWN
Unknown-building failures=1

```

```

--- SUMMARY ---
Audit entries: 9
Notifications sent: 2
All integrated tests completed.
==19544==
==19544== HEAP SUMMARY:
==19544==   in use at exit: 0 bytes in 0 blocks
==19544==   total heap usage: 105 allocs, 105 frees, 81,282 bytes allocated
==19544==
==19544== All heap blocks were freed -- no leaks are possible
==19544==
==19544== For lists of detected and suppressed errors, rerun with: -s
==19544== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

```

ajba@DESKTOP-CK66SOJ:/mnt/c/Users/Dell-Ajba/Music/214 prac 1$

```

```

CampusGuard down
==4345==
==4345== HEAP SUMMARY:
==4345==    in use at exit: 0 bytes in 0 blocks
==4345==   total heap usage: 109 allocs, 109 frees, 82,822 bytes allocated
==4345==
==4345== All heap blocks were freed -- no leaks are possible
==4345==
==4345== For lists of detected and suppressed errors, rerun with: -s
==4345== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
[root@06824793a2dda:/app# valgrind ./campusguard
==4346== Memcheck, a memory error detector
==4346== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==4346== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==4346== Command: ./campusguard
==4346==
  CAMPUSGUARD: EMERGENCY RESPONSE PLATFORM
Patterns: Command, Mediator, Adapter, Facade, State, Observer
|CAMPUSGUARD  MAIN MENU          |
|1  Run BOTH scenarios           |
|2  Scenario 1: Engineering lab  |
|3  Scenario 2: Library medical  |
|0  Quit                         |

```

During testing we investigated whether LockdownFacade::liftLockdown() correctly reports failure when called on a building with no registered areas.

```

Breakpoint 1, LockdownFacade::liftLockdown (this=0x7fffffffdc50, buildingId="GHOST", incidentRef="INC-TEST")
    at LockdownFacade.cpp:46
46                                     const std::string& incidentRef) {
#0  LockdownFacade::liftLockdown (this=0x7fffffffdc50, buildingId="GHOST", incidentRef="INC-TEST") at LockdownFacade.cpp:46
#1  0x0000555555555689e in main () at gdbtest.cpp:16
47      LockdownReport report;
48      audit_.record("LOCKDOWN LIFT building=" + buildingId + " incident=" + incidentRef);
[Audit #1] LOCKDOWN LIFT building=GHOST incident=INC-TEST
49      std::vector<Area> areas = access_.areasInBuilding(buildingId);
50      for (size_t i = 0; i < areas.size(); ++i) {
$1 = std::vector of length 0, capacity 0
$2 = 0
54      if (report.complete()) {
$3 = 0
$4 = 0
55      notifier_.broadcast("PA+SMS", "ALL CLEAR in " + buildingId);
$5 = true
[Notify:PA+SMS] ALL CLEAR in GHOST
[Audit #2] LOCKDOWN LIFTED building=GHOST
[Inferior 1 (process 1986) exited normally]

```

Cause: report.complete() only checks failed.empty() so zero areas read as success, the 1 line fix(areas.empty()) guard, matching initiateLockdown)

Contribution statements:

Ajba:

I was responsible for the Adapter and Facade patterns and the building-access part of CampusGuard. I designed and implemented the legacy integration: LegacyDoorPanel (the adaptee, which uses numeric zones, opcodes and integer return codes), the AccessController target interface, and LegacyPanelAdapter, which translates named areas and actions into panel calls and converts return codes into AccessFault exceptions. I also implemented BuildingAccessService (lock, unlock and restrict areas) and the LockdownFacade, whose initiateLockdown and liftLockdown operations coordinate the access service, NotificationService and AuditLog while leaving those subsystems independently usable. During testing I found and fixed a bug where liftLockdown broadcast an all-clear for a building that does not exist. I also ran Valgrind on the final integrated executable, which showed no leaks and no errors, and I wrote the Task 1 design, the Task 3 scenario and the Task 4 sequence diagram for my patterns.

Hannah:

I was responsible for the Command and Mediator design patterns, the associated UML work, Docker configuration and Valgrind memory testing.

For Command, I designed and implemented the Command hierarchy, including the concrete operator actions, and connected these commands to the OperatorConsole invoker and their appropriate receivers. For Mediator, I implemented the ResponseMediator / ResponseCoordinator design and the coordination between SecurityTeam, MedicalResponder, FacilitiesStaff and CommunicationService, ensuring that the response colleagues communicate through the mediator rather than directly with one another.

I also produced the UML material for the system. I configured the Docker environment used to build and launch the integrated CampusGuard application and ensured that the submitted system could be run using docker compose up --build. Finally, I ran Valgrind on the integrated executable and verified that all heap allocations were freed and that no memory errors were reported.

Lucian:

I was responsible for the two additional GoF patterns, State and Observer and for integrating all six patterns into one coherent application.

For State I designed and implemented the IncidentState hierarchy(ReportedState,DispatchedState,ContainedState and ResolvedState) so that Incident delegates its lifecycle transitions to its current state, while illegal transitions are rejected with a thrown IncidentTransitionError instead of being ignored.

For Observer I implemented IncidentObserver and IncidentBoard so that a valid state change on an Incident notifies the board and writes an entry to the shared AuditLog without Incident knowing.

I wrote the main wiring all the design patterns into a single CampusWorld and building 2 scenarios. An Engineering lab fire and a library medical emergency that each use all six

patterns. I also designed a behavioural diagram for the state pattern and did a GDB investigation into liftLockdown().

GitHub link:

https://github.com/u25044215-Hannah/COS214_Prac5/tree/main